

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 6
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Унарные и бинарные операции над графами»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.
Гуляевский Д.С.
Каташов К.А.

Приняли
Юрова О.В.
Деев М.В.

Пенза, 2025

Цель работы

Изучение и практическая реализация унарных и бинарных операций над графами, включая отождествление вершин, стягивание рёбер, объединение, пересечение, декартово произведение и кольцевую сумму, с использованием матриц смежности и списков смежности.

Лабораторное задание

Задание 1

1. Сгенерируйте (используя генератор случайных чисел) две матрицы M_1 , M_2 смежности неориентированных помеченных графов G_1 , G_2 . Выведите сгенерированные матрицы на экран.

2. * Для указанных графов преобразуйте представление матриц смежности в списки смежности. Выведите полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

2. * Для представления графов в виде списков смежности выполните операцию:

- а) отождествления вершин
- б) стягивания ребра
- в) расщепления вершины

Номера выбираемых для выполнения операции вершин ввести с клавиатуры.

Результат выполнения операции выведите на экран.

Задание 3

1. Для матричной формы представления графов выполните операцию:

- а) объединения $G = G_1 \cup G_2$
- б) пересечения $G = G_1 \cap G_2$
- в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции выведите на экран.

Задание 4 *

1. Для матричной формы представления графов выполните операцию декартова произведения графов $G = G_1 \times G_2$.

Результат выполнения операции выведите на экран.

Описание метода решения задачи

Для выполнения унарных операций (отождествление вершин, стягивание ребра, расщепление вершины) были разработаны алгоритмы, работающие с обоими представлениями. В случае матрицы смежности операции сводились к модификации строк и столбцов матрицы, объединению или разделению множеств смежности, с последующей корректировкой размерности матрицы. При работе со списками смежности операции выполнялись через манипуляции с динамическими списками соседей для каждой вершины, включая слияние списков при отождествлении, удаление и перенаправление ссылок. Для бинарных операций (объединение, пересечение, кольцевая сумма, декартово произведение) использовалось матричное представление, как наиболее наглядное для поэлементных логических операций. Объединение и пересечение реализованы через поэлементные операции ИЛИ и И над матрицами, кольцевая сумма – через исключающее ИЛИ. Декартово произведение выполнено по формальному определению: создана матрица размера $|V1| \cdot |V2|$, в которой ребро между вершинами (i,j) и (k,l) проводится, если $i=k$ и есть ребро (j,l) в G_2 , или если $j=l$ и есть ребро (i,k) в G_1 . Все алгоритмы включают проверку корректности входных данных (наличие вершин, ребер), а для наглядности реализован интерактивный ввод параметров операций и детальный вывод результатов.

Листинг

Задание 1

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>

int random_int(int min, int max) {
    return min + rand() % (max - min + 1);
}

int** generate_matrix(int vertices) {
    int** matrix = (int**)malloc(vertices * sizeof(int*));
```

```

for (int i = 0; i < vertices; i++) {
    matrix[i] = (int*)malloc(vertices * sizeof(int));
    for (int j = 0; j < vertices; j++) {
        matrix[i][j] = 0; // Инициализация нулями
    }
}

//заполнение матрицы
for (int i = 0; i < vertices; i++) {
    int edges_count = random_int(0, vertices - 1); // Максимум vertices-1 рёбер
    for (int j = 0; j < edges_count; j++) {
        int target_vertex = random_int(0, vertices - 1);
        if (target_vertex != i) { //исключаем петли
            matrix[i][target_vertex] = 1;
            matrix[target_vertex][i] = 1;
        }
    }
}

return matrix;
}

void print_matrix(int** matrix, int vertices, const char* name) {
    printf("\n%s (матрица смежности):\n", name);
    printf(" ");
    for (int i = 0; i < vertices; i++) {
        printf("%2d ", i);
    }
    printf("\n");

    for (int i = 0; i < vertices; i++) {
        printf("%2d ", i);
        for (int j = 0; j < vertices; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }
}

//преобразование в список
int** matrix_to_list(int** matrix, int vertices, int* list_sizes) {
    int** adj_list = (int**)malloc(vertices * sizeof(int*));

    for (int i = 0; i < vertices; i++) {
        //считаем количество соседей
        int neighbors_count = 0;
        for (int j = 0; j < vertices; j++) {
            if (matrix[i][j] == 1) {
                neighbors_count++;
            }
        }

        adj_list[i] = (int*)malloc(neighbors_count * sizeof(int));
        list_sizes[i] = neighbors_count;

        int index = 0;
        for (int j = 0; j < vertices; j++) {
            if (matrix[i][j] == 1) {
                adj_list[i][index++] = j;
            }
        }
    }
}

```

```

    }
}
}

return adj_list;
}

//Вывод списка
void print_list(int** adj_list, int* list_sizes, int vertices, const char* name) {
    printf("\n%s (список смежности):\n", name);
    for (int i = 0; i < vertices; i++) {
        printf("Вершина %d: ", i);
        if (list_sizes[i] == 0) {
            printf("нет соседей");
        }
        else {
            for (int j = 0; j < list_sizes[i]; j++) {
                printf("%d", adj_list[i][j]);
                if (j < list_sizes[i] - 1) {
                    printf(" ");
                }
            }
        }
        printf("\n");
    }
}

void free_matrix(int** matrix, int vertices) {
    for (int i = 0; i < vertices; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void free_list(int** adj_list, int* list_sizes, int vertices) {
    for (int i = 0; i < vertices; i++) {
        free(adj_list[i]);
    }
    free(adj_list);
    free(list_sizes);
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "rus");

    int vertices;
    printf("Введите количество вершин графа: ");
    scanf("%d", &vertices);

    if (vertices <= 0) {
        printf("Ошибка: количество вершин должно быть положительным числом!\n");
        return 1;
    }

    //Генерация
    int** M1 = generate_matrix(vertices);
    int** M2 = generate_matrix(vertices);

    //Вывод

```

```

print_matrix(M1, vertices, "Граф G1");
print_matrix(M2, vertices, "Граф G2");

//списки
int* list_sizes1 = (int*)malloc(vertices * sizeof(int));
int* list_sizes2 = (int*)malloc(vertices * sizeof(int));

int** adj_list1 = matrix_to_list(M1, vertices, list_sizes1);
int** adj_list2 = matrix_to_list(M2, vertices, list_sizes2);

//ВЫВОД СПИСКОВ
print_list(adj_list1, list_sizes1, vertices, "Граф G1");
print_list(adj_list2, list_sizes2, vertices, "Граф G2");

free_matrix(M1, vertices);
free_matrix(M2, vertices);
free_list(adj_list1, list_sizes1, vertices);
free_list(adj_list2, list_sizes2, vertices);

return 0;
}

```

Задание 2

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <iomanip>
#include <locale>
#include <random>

using namespace std;

class GraphMatrix {
private:
    vector<vector<int>>> matrix;
    int vertices;

public:
    GraphMatrix(int n) : vertices(n), matrix(n, vector<int>(n, 0)) {
        // Автоматическое создание рёбер с вероятностью 50%
        random_device rd;
        mt19937 gen(rd());
        uniform_real_distribution<> dis(0.0, 1.0);

        for (int i = 0; i < vertices; i++) {
            for (int j = i + 1; j < vertices; j++) {
                if (dis(gen) < 0.5) { // 50% вероятность
                    matrix[i][j] = 1;
                    matrix[j][i] = 1;
                }
            }
        }
    }

    void addEdge(int v1, int v2) {
        if (v1 >= 0 && v1 < vertices && v2 >= 0 && v2 < vertices && v1 != v2) {
            matrix[v1][v2] = 1;
            matrix[v2][v1] = 1;
        }
    }
}

```

```

void print(const string& name) {
    cout << "\n" << name << " (матрица смежности " << vertices << "x" << vertices << "):\n";
    cout << " ";
    for (int i = 0; i < vertices; i++) {
        cout << setw(2) << i + 1 << " ";
    }
    cout << "\n";

    for (int i = 0; i < vertices; i++) {
        cout << setw(2) << i + 1 << " ";
        for (int j = 0; j < vertices; j++) {
            cout << setw(2) << matrix[i][j] << " ";
        }
        cout << "\n";
    }
}

// а) Отождествление вершин
void identifyVertices(int v1, int v2) {
    if (!isValidVertex(v1) || !isValidVertex(v2) || v1 == v2) {
        cout << "Ошибка: неверные номера вершин!\n";
        return;
    }

    // Сохраняем вершину с меньшим номером, удаляем большую
    int keep = min(v1, v2);
    int remove = max(v1, v2);

    // Объединяем связи: keep получает все связи remove
    for (int i = 0; i < vertices; i++) {
        if (matrix[remove][i] == 1 && i != keep) {
            matrix[keep][i] = 1;
            matrix[i][keep] = 1;
        }
    }

    // Удаляем строку и столбец remove
    matrix.erase(matrix.begin() + remove);
    for (auto& row : matrix) {
        row.erase(row.begin() + remove);
    }

    vertices--;
    cout << "Вершины " << v1 + 1 << " и " << v2 + 1 << " отождествлены.\n";
}

// б) Стягивание ребра
void contractEdge(int v1, int v2) {
    if (!isValidVertex(v1) || !isValidVertex(v2) || v1 == v2) {
        cout << "Ошибка: неверные номера вершин!\n";
        return;
    }

    if (matrix[v1][v2] == 0) {
        cout << "Ошибка: вершины " << v1 + 1 << " и " << v2 + 1 << " не соединены ребром!\n";
        return;
    }

    identifyVertices(v1, v2);
    cout << "Ребро между вершинами " << v1 + 1 << " и " << v2 + 1 << " стянуто.\n";
}

```

```

}

// в) Расщепление вершины
void splitVertex(int v) {
    if (!isValidVertex(v)) {
        cout << "Ошибка: неверный номер вершины!\n";
        return;
    }

    // Добавляем новую строку и столбец
    for (auto& row : matrix) {
        row.push_back(0);
    }
    matrix.push_back(vector<int>(vertices + 1, 0));

    // Новая вершина
    int newVertex = vertices;

    // Переносим часть связей на новую вершину
    vector<int> connections;
    for (int i = 0; i < vertices; i++) {
        if (matrix[v][i] == 1 && i != v) {
            connections.push_back(i);
        }
    }

    // Переносим примерно половину связей
    for (size_t i = 0; i < connections.size(); i++) {
        if (i % 2 == 0) { // простой критерий для демонстрации
            int neighbor = connections[i];
            matrix[newVertex][neighbor] = 1;
            matrix[neighbor][newVertex] = 1;
            matrix[v][neighbor] = 0;
            matrix[neighbor][v] = 0;
        }
    }

    // Связываем новую вершину с исходной
    matrix[v][newVertex] = 1;
    matrix[newVertex][v] = 1;

    vertices++;
    cout << "Вершина " << v + 1 << " расщеплена на вершины " << v + 1 << " и " << newVertex
+ 1 << ".\n";
}

bool isValidVertex(int v) {
    return v >= 0 && v < vertices;
}

int getVerticesCount() { return vertices; }
};

class GraphList {
private:
    vector<vector<int>>> adjList;
    int vertices;

public:

```



```

GraphList(int n) : vertices(n), adjList(n) {
    // Автоматическое создание рёбер с вероятностью 50%
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<> dis(0.0, 1.0);

    for (int i = 0; i < vertices; i++) {
        for (int j = i + 1; j < vertices; j++) {
            if (dis(gen) < 0.5) { // 50% вероятность
                // Проверяем, нет ли уже такого ребра
                if (find(adjList[i].begin(), adjList[i].end(), j) == adjList[i].end()) {
                    adjList[i].push_back(j);
                }
                if (find(adjList[j].begin(), adjList[j].end(), i) == adjList[j].end()) {
                    adjList[j].push_back(i);
                }
            }
        }
    }
}

void addEdge(int v1, int v2) {
    if (v1 >= 0 && v1 < vertices && v2 >= 0 && v2 < vertices && v1 != v2) {
        // Проверяем, нет ли уже такого ребра
        if (find(adjList[v1].begin(), adjList[v1].end(), v2) == adjList[v1].end()) {
            adjList[v1].push_back(v2);
        }
        if (find(adjList[v2].begin(), adjList[v2].end(), v1) == adjList[v2].end()) {
            adjList[v2].push_back(v1);
        }
    }
}

void print(const string& name) {
    cout << "\n" << name << " (списки смежности):\n";
    for (int i = 0; i < vertices; i++) {
        cout << "Вершина " << i + 1 << ": ";
        if (adjList[i].empty()) {
            cout << "нет соседей";
        }
        else {
            for (size_t j = 0; j < adjList[i].size(); j++) {
                cout << adjList[i][j] + 1;
                if (j < adjList[i].size() - 1) {
                    cout << ", ";
                }
            }
        }
        cout << "\n";
    }
}

// а) Отождествление вершин
void identifyVertices(int v1, int v2) {
    if (!isValidVertex(v1) || !isValidVertex(v2) || v1 == v2) {
        cout << "Ошибка: неверные номера вершин!\n";
        return;
    }

    int keep = min(v1, v2);

```

```

int remove = max(v1, v2);

// Объединяем списки смежности
for (int neighbor : adjList[remove]) {
    if (neighbor != keep && find(adjList[keep].begin(), adjList[keep].end(), neighbor) ==
adjList[keep].end()) {
        adjList[keep].push_back(neighbor);
    }
}

// Обновляем ссылки у всех соседей remove
for (int i = 0; i < vertices; i++) {
    if (i != remove) {
        auto it = find(adjList[i].begin(), adjList[i].end(), remove);
        if (it != adjList[i].end()) {
            adjList[i].erase(it);
            if (i != keep && find(adjList[i].begin(), adjList[i].end(), keep) == adjList[i].end()) {
                adjList[i].push_back(keep);
            }
        }
    }
}

// Удаляем вершину remove
adjList.erase(adjList.begin() + remove);

// Обновляем номера вершин в списках
for (auto& list : adjList) {
    for (int& neighbor : list) {
        if (neighbor > remove) {
            neighbor--;
        }
    }
}

vertices--;
cout << "Вершины " << v1 + 1 << " и " << v2 + 1 << " отождествлены.\n";
}

// б) Стягивание ребра
void contractEdge(int v1, int v2) {
    if (!isValidVertex(v1) || !isValidVertex(v2) || v1 == v2) {
        cout << "Ошибка: неверные номера вершин!\n";
        return;
    }

    if (find(adjList[v1].begin(), adjList[v1].end(), v2) == adjList[v1].end()) {
        cout << "Ошибка: вершины " << v1 + 1 << " и " << v2 + 1 << " не соединены ребром!\n";
        return;
    }

    identifyVertices(v1, v2);
    cout << "Ребро между вершинами " << v1 + 1 << " и " << v2 + 1 << " стянуто.\n";
}

// в) Расщепление вершины
void splitVertex(int v) {
    if (!isValidVertex(v)) {
        cout << "Ошибка: неверный номер вершины!\n";
        return;
    }
}

```

```

    }

    // Добавляем новую вершину
    adjList.push_back(vector<int>());
    int newVertex = vertices;

    // Переносим часть связей на новую вершину
    vector<int> connections = adjList[v];
    for (size_t i = 0; i < connections.size(); i++) {
        if (i % 2 == 0) { // простой критерий для демонстрации
            int neighbor = connections[i];

            // Удаляем связь v-neighbor
            auto it = find(adjList[v].begin(), adjList[v].end(), neighbor);
            if (it != adjList[v].end()) {
                adjList[v].erase(it);
            }

            // Удаляем связь neighbor-v
            it = find(adjList[neighbor].begin(), adjList[neighbor].end(), v);
            if (it != adjList[neighbor].end()) {
                adjList[neighbor].erase(it);
            }

            // Добавляем связь newVertex-neighbor
            adjList[newVertex].push_back(neighbor);
            adjList[neighbor].push_back(newVertex);
        }
    }

    // Связываем новую вершину с исходной
    adjList[v].push_back(newVertex);
    adjList[newVertex].push_back(v);

    vertices++;
    cout << "Вершина " << v + 1 << " расщеплена на вершины " << v + 1 << " и " << newVertex
+ 1 << ".\n";
}

bool isValidVertex(int v) {
    return v >= 0 && v < vertices;
}

int getVerticesCount() { return vertices; }
};

int main() {
    setlocale(LC_ALL, "rus");

    int representation, vertices, choice;

    cout << "=== ОПЕРАЦИИ НАД ГРАФАМИ ===\n";

    cout << "Выберите представление графа:\n";
    cout << "1 - Матричное представление\n";
    cout << "2 - Списки смежности\n";
    cout << "Ваш выбор: ";
    cin >> representation;

    cout << "Введите количество вершин: ";

```

```

cin >> vertices;

if (vertices <= 0) {
    cout << "Ошибка: количество вершин должно быть положительным числом!\n";
    return 1;
}

if (representation == 1) {
    GraphMatrix graph(vertices);
    cout << "Создан граф с " << vertices << " вершинами и случайными рёбрами (вероятность
50%)\n";
    graph.print("Исходный граф");

    do {
        cout << "\nОперации:\n";
        cout << "1 - Добавить ребро\n";
        cout << "2 - Отождествить вершины\n";
        cout << "3 - Стянуть ребро\n";
        cout << "4 - Расщепить вершину\n";
        cout << "5 - Показать граф\n";
        cout << "0 - Выход\n";
        cout << "Ваш выбор: ";
        cin >> choice;

        int v1, v2;
        switch (choice) {
            case 1:
                cout << "Введите вершины ребра (v1 v2): ";
                cin >> v1 >> v2;
                graph.addEdge(v1 - 1, v2 - 1);
                break;
            case 2:
                cout << "Введите вершины для отождествления (v1 v2): ";
                cin >> v1 >> v2;
                graph.identifyVertices(v1 - 1, v2 - 1);
                graph.print("Граф после отождествления");
                break;
            case 3:
                cout << "Введите вершины ребра для стягивания (v1 v2): ";
                cin >> v1 >> v2;
                graph.contractEdge(v1 - 1, v2 - 1);
                graph.print("Граф после стягивания");
                break;
            case 4:
                cout << "Введите вершину для расщепления: ";
                cin >> v1;
                graph.splitVertex(v1 - 1);
                graph.print("Граф после расщепления");
                break;
            case 5:
                graph.print("Текущий граф");
                break;
            case 0:
                cout << "Выход из программы.\n";
                break;
            default:
                cout << "Неверный выбор!\n";
        }
    } while (choice != 0);
}

```

```

}
else {
    GraphList graph(vertices);
    cout << "Создан граф с " << vertices << " вершинами и случайными рёбрами.\n";
    graph.print("Исходный граф");

    do {
        cout << "\nОперации:\n";
        cout << "1 - Добавить ребро\n";
        cout << "2 - Отождествить вершины\n";
        cout << "3 - Стянуть ребро\n";
        cout << "4 - Расщепить вершину\n";
        cout << "5 - Показать граф\n";
        cout << "0 - Выход\n";
        cout << "Ваш выбор: ";
        cin >> choice;

        int v1, v2;
        switch (choice) {
            case 1:
                cout << "Введите вершины ребра (v1 v2): ";
                cin >> v1 >> v2;
                graph.addEdge(v1 - 1, v2 - 1);
                break;
            case 2:
                cout << "Введите вершины для отождествления (v1 v2): ";
                cin >> v1 >> v2;
                graph.identifyVertices(v1 - 1, v2 - 1);
                graph.print("Граф после отождествления");
                break;
            case 3:
                cout << "Введите вершины ребра для стягивания (v1 v2): ";
                cin >> v1 >> v2;
                graph.contractEdge(v1 - 1, v2 - 1);
                graph.print("Граф после стягивания");
                break;
            case 4:
                cout << "Введите вершину для расщепления: ";
                cin >> v1;
                graph.splitVertex(v1 - 1);
                graph.print("Граф после расщепления");
                break;
            case 5:
                graph.print("Текущий граф");
                break;
            case 0:
                cout << "Выход из программы.\n";
                break;
            default:
                cout << "Неверный выбор!\n";
        }
    } while (choice != 0);
}

return 0;
}

```

Задание 3

```

#include <iostream>
#include <vector>
#include <iomanip>

```

```

#include <locale>
#include <random>

using namespace std;

class GraphMatrix {
private:
    vector<vector<int>> matrix;
    int vertices;

public:
    // Конструктор со случайным заполнением
    GraphMatrix(int n) : vertices(n), matrix(n, vector<int>(n, 0)) {
        random_device rd;
        mt19937 gen(rd());
        uniform_real_distribution<> dis(0.0, 1.0);

        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                // Для петель (i == j) и рёбер (i != j) вероятность 50%
                if (dis(gen) < 0.5) {
                    matrix[i][j] = 1;
                    // Для неориентированного графа симметрично
                    if (i != j) {
                        matrix[j][i] = 1;
                    }
                }
            }
        }
    }

    void print(const string& name) {
        cout << "\n" << name << " (матрица смежности " << vertices << "x" << vertices << "):\n";
        cout << " ";
        for (int i = 0; i < vertices; i++) {
            cout << setw(2) << i + 1 << " ";
        }
        cout << "\n";

        for (int i = 0; i < vertices; i++) {
            cout << setw(2) << i + 1 << " ";
            for (int j = 0; j < vertices; j++) {
                cout << setw(2) << matrix[i][j] << " ";
            }
            cout << "\n";
        }

        // Подсчет петель и рёбер для информации
        int loops = 0;
        int edges = 0;
        for (int i = 0; i < vertices; i++) {
            for (int j = 0; j < vertices; j++) {
                if (matrix[i][j] == 1) {
                    if (i == j) {
                        loops++;
                    }
                    else if (i < j) { // Считаем каждое ребро только один раз
                        edges++;
                    }
                }
            }
        }
    }
}

```

```

    }
}
cout << "Петли: " << loops << ", Рёбра: " << edges << "\n";
}

// а) Объединение графов  $G = G1 \cup G2$ 
static GraphMatrix unionGraphs(const GraphMatrix& g1, const GraphMatrix& g2) {
    if (g1.vertices != g2.vertices) {
        throw invalid_argument("Графы должны иметь одинаковое количество вершин!");
    }

    int n = g1.vertices;
    GraphMatrix result(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Объединение: ребро есть, если оно есть в G1 ИЛИ в G2
            result.matrix[i][j] = g1.matrix[i][j] || g2.matrix[i][j];
        }
    }

    return result;
}

// б) Пересечение графов  $G = G1 \cap G2$ 
static GraphMatrix intersectionGraphs(const GraphMatrix& g1, const GraphMatrix& g2) {
    if (g1.vertices != g2.vertices) {
        throw invalid_argument("Графы должны иметь одинаковое количество вершин!");
    }

    int n = g1.vertices;
    GraphMatrix result(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Пересечение: ребро есть, если оно есть в G1 И в G2
            result.matrix[i][j] = g1.matrix[i][j] && g2.matrix[i][j];
        }
    }

    return result;
}

// в) Кольцевая сумма  $G = G1 \oplus G2$ 
static GraphMatrix ringSumGraphs(const GraphMatrix& g1, const GraphMatrix& g2) {
    if (g1.vertices != g2.vertices) {
        throw invalid_argument("Графы должны иметь одинаковое количество вершин!");
    }

    int n = g1.vertices;
    GraphMatrix result(n);

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            // Кольцевая сумма: ребро есть, если оно есть в G1 ИЛИ в G2, но не в обоих
            result.matrix[i][j] = g1.matrix[i][j] != g2.matrix[i][j];
        }
    }

    return result;
}

```

```

    }

    int getVerticesCount() const { return vertices; }
};

int main() {
    setlocale(LC_ALL, "rus");

    cout << "Объединение, пересечение и кольцевая сумма\n\n";

    int vertices;
    cout << "Введите количество вершин для графов: ";
    cin >> vertices;

    if (vertices <= 0) {
        cout << "Ошибка: количество вершин должно быть положительным числом!\n";
        return 1;
    }

    // Создаем два случайных графа
    cout << "\nСоздание графов со случайными рёбрами и петлями (вероятность 50%)...\n";

    GraphMatrix g1(vertices);
    GraphMatrix g2(vertices);

    g1.print("Граф G1");
    g2.print("Граф G2");

    // Автоматически выполняем все операции
    try {
        // Объединение
        GraphMatrix unionResult = GraphMatrix::unionGraphs(g1, g2);
        unionResult.print("Объединение G1 G2");

        // Пересечение
        GraphMatrix intersectionResult = GraphMatrix::intersectionGraphs(g1, g2);
        intersectionResult.print("Пересечение G1 G2");

        // Кольцевая сумма
        GraphMatrix ringSumResult = GraphMatrix::ringSumGraphs(g1, g2);
        ringSumResult.print("Кольцевая сумма G1 G2");

    }
    catch (const invalid_argument& e) {
        cout << "Ошибка: " << e.what() << endl;
    }

    return 0;
}

```

Задание 4

```

#include <iostream>
#include <vector>
#include <iomanip>
#include <random>

using namespace std;

class Graph {
private:
    vector<vector<int>> matrix;

```



```

int vertices;

public:
    // Конструктор
    Graph(int n, bool randomize = false) : vertices(n), matrix(n, vector<int>(n, 0)) {
        if (randomize) {
            random_device rd;
            mt19937 gen(rd());
            uniform_real_distribution<> dis(0.0, 1.0);

            // Создаем случайные ребра с вероятностью 40%
            for (int i = 0; i < vertices; i++) {
                for (int j = i + 1; j < vertices; j++) {
                    if (dis(gen) < 0.4) {
                        matrix[i][j] = 1;
                        matrix[j][i] = 1;
                    }
                }
            }
        }
    }

    // Добавление ребра
    void addEdge(int v1, int v2) {
        if (v1 >= 0 && v1 < vertices && v2 >= 0 && v2 < vertices && v1 != v2) {
            matrix[v1][v2] = 1;
            matrix[v2][v1] = 1;
        }
    }

    // Получение значения ребра
    int getEdge(int i, int j) const {
        if (i >= 0 && i < vertices && j >= 0 && j < vertices) {
            return matrix[i][j];
        }
        return 0;
    }

    // Получение количества вершин
    int getVerticesCount() const {
        return vertices;
    }

    // Вывод матрицы смежности
    void print(const string& name) const {
        cout << "\n" << name << " (матрица смежности " << vertices << "x" << vertices << "):\n";

        // Заголовок столбцов
        cout << " ";
        for (int i = 0; i < vertices; i++) {
            cout << setw(2) << i + 1 << " ";
        }
        cout << "\n";

        // Строки матрицы
        for (int i = 0; i < vertices; i++) {
            cout << setw(2) << i + 1 << " ";
            for (int j = 0; j < vertices; j++) {
                cout << setw(2) << matrix[i][j] << " ";
            }
        }
    }

```



```

    }
};

// Функция для вывода графа с понятными именами вершин
void printCartesianResult(const Graph& graph, int n2, const string& name) {
    int vertices = graph.getVerticesCount();
    cout << "\n" << name << ":\n";
    cout << "Формат вершин: (номер_в_G1, номер_в_G2)\n";
    cout << "Список смежности:\n";

    for (int i = 0; i < vertices; i++) {
        string vertexName = Graph::getVertexName(i, n2);
        cout << vertexName << ": ";

        bool hasNeighbors = false;
        for (int j = 0; j < vertices; j++) {
            if (graph.getEdge(i, j) == 1 && i != j) {
                if (hasNeighbors) cout << ", ";
                cout << Graph::getVertexName(j, n2);
                hasNeighbors = true;
            }
        }

        if (!hasNeighbors) {
            cout << "нет соседей";
        }
        cout << "\n";
    }
}

int main() {
    // Установка локали для русского языка
    setlocale(LC_ALL, "rus");

    cout << "=== ДЕКАРТОВО ПРОИЗВЕДЕНИЕ ГРАФОВ ===\n\n";

    int n1, n2;

    // Ввод размеров графов
    cout << "Введите количество вершин в графе G1: ";
    cin >> n1;
    cout << "Введите количество вершин в графе G2: ";
    cin >> n2;

    if (n1 <= 0 || n2 <= 0) {
        cout << "Ошибка: количество вершин должно быть положительным!\n";
        return 1;
    }

    // Создание случайных графов
    cout << "\nСоздание графов со случайными ребрами...\n";
    Graph g1(n1, true);
    Graph g2(n2, true);

    // Вывод исходных графов
    g1.print("Граф G1");
    g2.print("Граф G2");

    // Выполнение декартова произведения
    cout << "\n=== ВЫПОЛНЕНИЕ ДЕКАРТОВА ПРОИЗВЕДЕНИЯ ===\n";
}

```

```
Graph result = Graph::cartesianProduct(g1, g2);  
  
// Вывод результатов  
result.print("Результат G1 X G2 (матрица смежности)");  
  
return 0;  
}
```

Результаты работы программы

Задание 1

```

C:\> Консоль отладки Microsoft Visual Studio
Введите количество вершин графа: 4

Граф G1 (матрица смежности):
  0  1  2  3
0  0  1  1  1
1  1  0  0  1
2  1  0  0  1
3  1  1  1  0

Граф G2 (матрица смежности):
  0  1  2  3
0  0  1  1  0
1  1  0  1  1
2  1  1  0  1
3  0  1  1  0

Граф G1 (список смежности):
Вершина 0: 1, 2, 3
Вершина 1: 0, 3
Вершина 2: 0, 3
Вершина 3: 0, 1, 2

Граф G2 (список смежности):
Вершина 0: 1, 2
Вершина 1: 0, 2, 3
Вершина 2: 0, 1, 3
Вершина 3: 1, 2

```

Задание 2

```

C:\> E:\rep\6lab\6lab2\x64\Debug\6lab2.exe
=== ОПЕРАЦИИ НАД ГРАФАМИ ===
Выберите представление графа:
1 - Матричное представление
2 - Списки смежности
Ваш выбор: 1
Введите количество вершин: 5
Создан граф с 5 вершинами и случайными рёбрами (вероятность 50%).

Исходный граф (матрица смежности 5x5):
  1  2  3  4  5
1  0  1  1  0  0
2  1  0  0  1  1
3  1  0  0  0  1
4  0  1  0  0  1
5  0  1  1  1  0

Операции:
1 - Добавить ребро
2 - отождествить вершины
3 - Стянуть ребро
4 - Расщепить вершину
5 - Показать граф
0 - Выход
Ваш выбор: 4
Введите вершину для расщепления: 1
Вершина 1 расщеплена на вершины 1 и 6.

Граф после расщепления (матрица смежности 6x6):
  1  2  3  4  5  6
1  0  0  1  0  0  1
2  0  0  0  1  1  1
3  1  0  0  0  0  1  0
4  0  1  0  0  0  1  0
5  0  1  1  1  0  0  0
6  1  1  0  0  0  0  0

Операции:
1 - Добавить ребро
2 - отождествить вершины
3 - Стянуть ребро
4 - Расщепить вершину
5 - Показать граф
0 - Выход
Ваш выбор: _

```

Задание 3

Консоль отладки Microsoft Visual Studio

Объединение, пересечение и кольцевая сумма

Введите количество вершин для графов: 5

Создание графов со случайными рёбрами и петлями (вероятность 50%)...

Граф G1 (матрица смежности 5x5):

	1	2	3	4	5
1	0	1	1	0	1
2	1	1	1	1	1
3	1	1	1	1	1
4	0	1	1	0	1
5	1	1	1	1	1

Петли: 3, Рёбра: 9

Граф G2 (матрица смежности 5x5):

	1	2	3	4	5
1	0	1	1	0	0
2	1	1	1	1	1
3	1	1	0	1	1
4	0	1	1	0	0
5	0	1	1	0	0

Петли: 1, Рёбра: 7

Объединение G1 G2 (матрица смежности 5x5):

	1	2	3	4	5
1	0	1	1	0	1
2	1	1	1	1	1
3	1	1	1	1	1
4	0	1	1	0	1
5	1	1	1	1	1

Петли: 3, Рёбра: 9

Пересечение G1 G2 (матрица смежности 5x5):

	1	2	3	4	5
1	0	1	1	0	0
2	1	1	1	1	1
3	1	1	0	1	1
4	0	1	1	0	0
5	0	1	1	0	0

Петли: 1, Рёбра: 7

Кольцевая сумма G1 G2 (матрица смежности 5x5):

	1	2	3	4	5
1	0	0	0	0	1
2	0	0	0	0	0
3	0	0	1	0	0
4	0	0	0	0	1
5	1	0	0	1	1

Петли: 2, Рёбра: 2

E:\ger\6lab\6lab3\x64\Debug\6lab3.exe (процесс 1092) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

Задание 4

cs Консоль отладки Microsoft Visual Studio

=== ДЕКАРТОВО ПРОИЗВЕДЕНИЕ ГРАФОВ ===

Введите количество вершин в графе G1: 5

Введите количество вершин в графе G2: 5

Создание графов со случайными ребрами...

Граф G1 (матрица смежности 5x5):

	1	2	3	4	5
1	0	0	1	0	1
2	0	0	1	0	0
3	1	1	0	0	0
4	0	0	0	0	1
5	1	0	0	1	0

Количество ребер: 4

Граф G2 (матрица смежности 5x5):

	1	2	3	4	5
1	0	1	0	0	0
2	1	0	0	1	1
3	0	0	0	0	1
4	0	1	0	0	1
5	0	1	1	1	0

Количество ребер: 5

=== ВЫПОЛНЕНИЕ ДЕКАРТОВА ПРОИЗВЕДЕНИЯ ===

Результат G1 X G2 (матрица смежности) (матрица смежности 25x25):

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0
2	1	0	0	1	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0
3	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0
4	0	1	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0
5	0	1	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1
6	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	1	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
10	0	0	0	0	0	0	1	1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
11	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
12	0	1	0	0	0	1	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0
13	0	0	1	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
14	0	0	0	1	0	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0
15	0	0	0	0	1	0	0	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0
17	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	1	0	1	0	0	0
18	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0
19	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	1	0
20	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	1
21	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0
22	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	1	1
23	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1
24	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	1
25	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1	1	0

Количество ребер: 45

E:\гер\6lab\6lab4\х64\Debug\6lab4.exe (процесс 8932) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:

Выводы

В ходе выполнения лабораторной работы были успешно изучены и реализованы основные унарные и бинарные операции над графами, такие как отождествление вершин, стягивание ребра, расщепление вершины, объединение, пересечение, кольцевая сумма и декартово произведение. Практическое применение двух форм представления графов — матриц смежности и списков смежности — позволило оценить их преимущества и недостатки: матричное представление оказалось более удобным для бинарных операций и наглядного отображения структуры графа, в то время как списки смежности продемонстрировали эффективность при работе с разреженными графами и выполнении операций, связанных с изменением множества вершин. Разработанные программы корректно выполняют все требуемые операции, включая обработку ошибочных входных данных и визуализацию результатов. Работа подтвердила важность выбора адекватной структуры данных в зависимости от решаемой задачи и закрепила понимание теоретических основ операций над графами через их практическую реализацию.